

Project Explanation -- AI Inventory Agent

A step-by-step walkthrough of how the system works, module by module.

1. System Overview

The AI Inventory Agent is a three-part system:

1. Frontend (Bootstrap 5.3) -- What the user sees and interacts with
2. Backend (Spring Boot) -- REST API handling all business logic and database access
3. AI Service (Flask/Python) -- Independent microservice for AI analytics

```
Browser -> Frontend (port 80) -> Backend (port 8080) -> MySQL
                                     |
                                     AI Service (port 5000)
```

2. Authentication Module

How Login Works

Step 1: User enters username/password on login.html

```
// frontend/js/auth.js
apiRequest('/auth/login', {
  method: 'POST',
  body: JSON.stringify({ username, password })
})
```

Step 2: Backend receives at POST /api/auth/login

```
// AuthService.java
public AuthResponse login(LoginRequest request) {
    User user = userRepository.findByUsername(request.getUsername())
        .orElseThrow(() -> new RuntimeException("Invalid username or password"));

    if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
        throw new RuntimeException("Invalid username or password");
    }

    String token = generateToken(user.getUsername());
    return AuthResponse.builder()
        .token(token)
        .username(savedUser.getUsername())
        .role(savedUser.getRole())
        .build();
}
```

Step 3: JWT token generated and returned

```
private String generateToken(String username) {
    return Jwts.builder()
        .subject(username)
        .claim("username", username)
        .issuedAt(now)
        .expiration(expiryDate) // 24 hours
        .signWith(key, SignatureAlgorithm.HS256)
        .compact();
}
```

Step 4: Frontend stores token in sessionStorage

```
sessionStorage.setItem('token', data.token);
sessionStorage.setItem('user', JSON.stringify(data));
window.location.href = 'dashboard.html';
```

Step 5: Every subsequent API call includes the token

```
// common.js - apiRequest() automatically adds:
headers['Authorization'] = 'Bearer ' + getToken();
```

Step 6: JwtAuthenticationFilter validates on every request

```
// SecurityConfig.java - JwtAuthenticationFilter
String jwt = authHeader.substring(7);
String username = jwtUtil.extractUsername(jwt);
if (jwtUtil.validateToken(jwt, userDetails)) {
    // Set authentication context
    SecurityContextHolder.getContext().setAuthentication(authToken);
}
```

Key Security Properties

- Stateless sessions (no server-side session)
- Passwords hashed with BCrypt
- Token expires after 24 hours
- Browser close clears sessionStorage = logout

3. Products Module

CRUD Operations

Create Product:

Frontend sends to POST /api/products:

```
{
  "name": "Wireless Mouse",
  "sku": "ELEC-001",
  "category": "Electronics",
  "unitPrice": 29.99,
  "costPrice": 15.00,
  "reorderLevel": 20
}
```

Controller receives, validates, passes to service:

```
@PostMapping
public ResponseEntity<Product> createProduct(@RequestBody ProductRequest
    request) {
    return ResponseEntity.ok(productService.createProduct(request));
}
```

Service creates entity with timestamps:

```
public Product createProduct(ProductRequest request) {
    Product product = new Product();
    product.setName(request.getName());
    product.setSku(request.getSku());
    product.setUnitPrice(request.getUnitPrice());
    // ... other fields
    product.setIsActive(true);
    return productRepository.save(product);
}
```

Toggle Active/Inactive:

```
@PatchMapping("/{id}/toggle-status")
public ResponseEntity<Product> toggleProductStatus(@PathVariable Long id
) {
    return ResponseEntity.ok(productService.toggleProductStatus(id));
}
```

```
public Product toggleProductStatus(Long id) {
    Product product = productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Product not found"));

    product.setIsActive(!product.getIsActive());
    return productRepository.save(product);
}
```

Frontend shows green toggle button in table:

```
function toggleProductStatus(productId) {
    apiRequest(`/products/${productId}/toggle-status`, { method: 'PATCH'
    })
        .then(() => loadProducts());
}
```

Product Filtering

```
// products.js
function applyProductFilters() {
    const category = document.getElementById('filterCategory').value;
    const status = document.getElementById('filterStatus').value;
    const sort = document.getElementById('filterSort').value;

    let filtered = [...allProducts];

    if (category) filtered = filtered.filter(p => p.category === category);
    if (status === 'active') filtered = filtered.filter(p => p.isActive);
    if (status === 'inactive') filtered = filtered.filter(p => !p.isActive);

    if (sort === 'name') filtered.sort((a, b) => a.name.localeCompare(b.name));
    if (sort === 'price-asc') filtered.sort((a, b) => a.unitPrice - b.unitPrice);

    renderProducts(filtered);
}
```

4. Inventory Module

Stock Movement Recording

When a user records a stock movement:

```
POST /api/inventory/movements
{
  "productId": 1,
  "movementType": "IN",
  "quantity": 100,
  "notes": "Initial stock"
}
```

The controller captures the authenticated user:

```
@PostMapping("/movements")
```

```
public ResponseEntity<InventoryMovement> createMovement(@RequestBody Map
<String, Object> movementRequest) {
    Authentication auth = SecurityContextHolder.getContext().getAuthenti
cation();
    String username = auth != null ? auth.getName() : "system";
    return ResponseEntity.ok(inventoryService.createMovement(movementReq
uest, username));
}
```

Stock Calculation

Stock is calculated on-the-fly from movements:

```
# AI service reads movements to calculate stock
def calculate_current_stock(product_id, movements):
    stock = 0
    for _, movement in product_movements.iterrows():
        if movement['movement_type'] == 'IN':
            stock += movement['quantity']
        elif movement['movement_type'] == 'OUT':
            stock -= movement['quantity']
        elif movement['movement_type'] == 'ADJUST':
            stock += movement['quantity']
    return stock
```

5. Sales Module

Recording a Sale

```
POST /api/sales
{
    "productId": 1,
    "quantity": 5,
    "unitPrice": 29.99,
    "customerName": "John Doe"
}
```

The entity auto-calculates total and sets defaults:

```
@PrePersist
protected void onCreate() {
    this.saleDate = LocalDateTime.now();
    this.status = "PENDING";
    calculateTotalPrice(); // unitPrice * quantity
}

private void calculateTotalPrice() {
    this.totalPrice = this.unitPrice.multiply(BigDecimal.valueOf(this.qu
antity));
}
```

Sales Filtering

Frontend filters happen client-side after fetching all sales:

```
function filterSales() {
    const customer = document.getElementById('filterCustomer').value.toL
owerCase();
    const product = document.getElementById('filterProduct').value;
    const status = document.getElementById('filterStatus').value;
    const startDate = document.getElementById('filterStartDate').value;
    const endDate = document.getElementById('filterEndDate').value;

    let filtered = [...allSales];

    if (customer) filtered = filtered.filter(s =>
        s.customerName?.toLowerCase().includes(customer));
```

```

    if (product) filtered = filtered.filter(s =>
      s.product?.id == product);
    if (status) filtered = filtered.filter(s => s.status === status);
    if (startDate) filtered = filtered.filter(s =>
      new Date(s.saleDate) >= new Date(startDate));

    renderSales(filtered);
  }

```

6. Purchases Module

Purchase Order Lifecycle

ORDERED -> RECEIVED (stock increases)
 ORDERED -> CANCELLED (no stock change)

Receive a Purchase:

```

function receivePurchase(purchaseId) {
  // Records an IN movement for the product quantity
  apiRequest('/inventory/movements', {
    method: 'POST',
    body: JSON.stringify({
      productId: purchase.product.id,
      movementType: 'IN',
      quantity: purchase.quantity,
      referenceNumber: 'PO-' + purchaseId,
      notes: 'Stock received from purchase order'
    })
  }).then(() => {
    // Update purchase status to RECEIVED
    apiRequest(`/purchases/${purchaseId}`, {
      method: 'PUT',
      body: JSON.stringify({ ...purchase, status: 'RECEIVED' })
    });
  });
}

```

Purchase Filtering

```

function applyPurchaseFilters() {
  const supplier = document.getElementById('filterSupplier').value;
  const status = document.getElementById('filterStatus').value;
  const startDate = document.getElementById('filterStartDate').value;
  const endDate = document.getElementById('filterEndDate').value;

  let filtered = [...allPurchases];

  if (supplier) filtered = filtered.filter(p => p.supplier?.id == supplier);
  if (status) filtered = filtered.filter(p => p.status === status);
  if (startDate) filtered = filtered.filter(p =>
    new Date(p.purchaseDate) >= new Date(startDate));

  renderPurchases(filtered);
}

```

7. Dashboard Module

Data Loading

```

// dashboard.js
async function loadDashboard() {

```

```

const stats = await apiRequest('/dashboard/stats');

document.getElementById('totalProducts').textContent = stats.totalPr
oducts;
document.getElementById('totalSales').textContent = stats.totalSales
;
document.getElementById('totalRevenue').textContent = formatCurrency
(stats.totalRevenue);

renderSalesTrendChart(stats.salesTrend);
renderCategoryChart(stats.categoryDistribution);
renderTopProductsChart(stats.topProducts);
renderRecentActivity(stats.recentActivity);
}

```

Charts (Chart.js)

Sales Trend Line Chart:

```

function renderSalesTrendChart(data) {
  new Chart(document.getElementById('salesTrendChart'), {
    type: 'line',
    data: {
      labels: data.map(d => d.date),
      datasets: [{
        label: 'Sales',
        data: data.map(d => d.total),
        borderColor: '#4f46e5',
        fill: true
      }]
    }
  });
}

```

8. AI Service Module

Flask Application Entry

```

# ai-service/app.py
app = Flask(__name__)
CORS(app)

@app.route('/api/ai/forecast/<int:product_id>', methods=['GET'])
def get_forecast(product_id):
    result = demand_forecast(product_id)
    return jsonify(result)

```

Demand Forecasting Flow

1. Read sales_data.csv for the product
2. If fewer than 3 records -> use moving average
3. If 3+ records -> attempt ML model prediction
4. Generate 7-day forecast with dates and quantities
5. Return JSON with method used and confidence scores

Moving Average:

```

recent_avg = product_sales['quantity'].tail(3).mean()
predicted_qty = max(0, int(recent_avg * (1 + np.random.uniform(-0.1, 0.1
))))

```

ML Model:

```

features = [[weekday, month, avg_unit_price, avg_quantity]]
features_scaled = scaler.transform(features)
predicted_qty = model.predict(features_scaled)[0]

```

Stock Health Scoring

```
For each product:
    stock_ratio = current_stock / reorder_level
    days_of_stock = current_stock / sales_velocity

    stock_score = f(stock_ratio)    # 60% weight
    velocity_score = f(days_of_stock) # 40% weight

    health_score = stock_score * 0.6 + velocity_score * 0.4

    Status: healthy (>=80), warning (50-79), critical (<50)
```

Anomaly Detection

Uses Z-score statistical analysis:

```
z_score = (value - mean) / standard_deviation

if abs(z_score) > 2.5:
    anomaly_detected (medium severity)
if abs(z_score) > 3.0:
    anomaly_detected (high severity)
```

Three detection types:

- Sales quantity anomalies per product
- Price anomalies per product
- Daily sales velocity anomalies

Reorder Engine

```
def calculate_reorder_quantity(current_stock, sales_velocity, reorder_level):
    safety_stock = reorder_level * 0.5
    lead_time = 7 # days
    reorder_qty = (sales_velocity * lead_time) + safety_stock - current_stock
    return max(0, reorder_qty)
```

Urgency based on days until stockout:

- critical: <= 3 days
- warning: 4-7 days
- normal: > 7 days

Natural Language Query

```
def handle_natural_language_query(query):
    if 'forecast' in query:
        return {'type': 'forecast', 'message': 'Use /api/ai/forecast/<id>'}
    elif 'reorder' in query:
        return {'type': 'reorder', 'data': reorder_engine()}
    elif 'anomal' in query:
        return {'type': 'anomalies', 'data': anomaly_detection()}
    elif 'health' in query:
        return {'type': 'stock_health', 'data': stock_health()}
```

9. Frontend Common Utilities

common.js - Shared Functions

Function	Purpose
`getToken()`	Get JWT from sessionStorage
`getUser()`	Get user object from sessionStorage
`setAuth(token, user)`	Store auth data
`logout()`	Clear storage, redirect to login
`requireAuth()`	Redirect to login if no token
`apiRequest(endpoint, options)`	Fetch with JWT header
`formatDate(dateStr)`	Format date string
`formatCurrency(amount)`	Format as \$X.XX
`showToast(message, type)`	Show notification toast
`openModal(id)` / `closeModal(id)`	Toggle modal visibility
`toggleSidebar()`	Mobile sidebar toggle
`toggleFilter(panelId)`	Collapsible filter panel

Modal System

```
/* style.css */
.modal { display: block !important; }
.modal-overlay {
  display: none;
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.5);
  z-index: 1050;
}
.modal-overlay.active { display: flex; }
```

```
function openModal(id) {
  document.getElementById(id).classList.add('active');
}
function closeModal(id) {
  document.getElementById(id).classList.remove('active');
}
```

10. Data Flow Summary

Adding a New Sale

1. User fills form on sales.html
2. sales.js calls apiRequest('/sales', { POST, body })
3. Backend: SalesController.createSale()
4. Backend: SalesService.createSale()
5. Backend: Calculates totalPrice, sets PENDING status
6. Backend: saleRepository.save() -> MySQL INSERT
7. Backend: Returns Sale JSON
8. Frontend: Reloads sales table

Recording Stock Movement

1. User fills form on inventory.html
2. inventory.js calls apiRequest('/inventory/movements', { POST })
3. Backend: InventoryController.createMovement()
4. Backend: Captures authenticated username
5. Backend: inventoryService.createMovement()
6. Backend: movementRepository.save() -> MySQL INSERT
7. Frontend: Reloads movements table and stock levels

AI Forecast Request

1. User selects product on ai-agent.html

```
2. ai-agent.js calls fetch('http://localhost:5000/api/ai/forecast/1')
3. Flask: get_forecast(1)
4. Flask: demand_forecast(1)
5. Flask: Reads sales_data.csv, filters by product_id
6. Flask: Applies moving average or ML model
7. Flask: Returns 7-day forecast JSON
8. Frontend: Renders forecast chart
```

11. Configuration

Backend (application.properties)

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/inventory_db
jwt.secret=... (Base64 encoded)
jwt.expiration-ms=86400000 (24 hours)
ai.service.url=http://localhost:5000
```

Frontend (config.js)

```
var CONFIG = {
  API_BASE: 'http://localhost:8080/api'
};
```

AI Service (app.py)

```
app.run(debug=False, port=5000)
```

12. File Count Summary

Component	Files
Backend Java	30 files (entities, services, controllers, configs)
AI Service Python	5 files (app.py + 4 service modules)
Frontend HTML	11 pages
Frontend JS	9 modules
Frontend CSS	4 stylesheets
Data files	4 CSV files
Total	**63 source files**